

Project Development Journal

Day 01 — 14 June 2026

Project Goal

Upgrade a basic client-server messaging application into a scalable multi-client chat system capable of handling concurrent connections and broadcasting messages.

Initial State

At the beginning of the project:

- Server could communicate with only one client.
 - Client could send messages to the server.
 - Server could not accept multiple client connections simultaneously.
 - Broadcasting functionality was not available.
 - No client management system existed.
-

Task 1: Enable Multiple Client Connections

Problem

Initially, the server accepted only one client connection:

```
Socket socket = serverSocket.accept();
```

Since `accept()` is a blocking operation, the server remained occupied with that connection and could not accept additional clients.

Solution

Implemented a continuous listening loop:

```
while(true){  
    Socket socket = serverSocket.accept();  
  
    ClientHandler clientHandler =  
        new ClientHandler(socket);
```

```
        threadPool.execute(clientHandler);  
    }  
}
```

This allows the server to continuously listen for and accept new client connections.

What I Learned

ServerSocket.accept()

- Waits for a client connection.
- It is a blocking operation.
- Execution pauses until a client connects.

Why a While Loop Is Needed

Without a loop:

- Only one client can connect.

With a loop:

- The server continuously listens.
 - New clients can connect at any time.
-

Result

✓ Server can now accept multiple client connections.

Task 2: Implement Thread Pool

Problem

Creating a new thread manually for every client connection can become inefficient as the number of clients increases.

Example:

```
new Thread(clientHandler).start();
```

Problems with this approach:

- Excessive thread creation.
- Increased memory usage.
- Higher CPU overhead due to context switching.

- Difficult thread management.
-

Solution

Implemented a thread pool using `ExecutorService`.

```
ExecutorService threadPool =  
    Executors.newCachedThreadPool();
```

Each client handler is submitted to the pool:

```
threadPool.execute(clientHandler);
```

What I Learned

ExecutorService

A Java framework used to manage threads efficiently.

Thread Pool

A collection of reusable worker threads.

Instead of creating a new thread every time:

1. A task is submitted.
2. An available thread executes it.
3. The thread is reused after completion.

Why `newCachedThreadPool()`

- Creates threads when required.
 - Reuses idle threads.
 - Automatically expands for increased load.
 - Improves performance and resource management.
-

Benefits

- Better performance
- Lower memory consumption
- Cleaner code
- Efficient thread management
- Improved scalability

Result

✓ Multiple clients can now be handled efficiently through a managed thread pool.

Task 3: Create ClientHandler

Problem

Even if the server accepts multiple clients, a single execution thread cannot communicate with all clients simultaneously.

Solution

Created a dedicated ClientHandler class.

```
class ClientHandler implements Runnable
```

Each connected client receives its own execution thread.

Why ClientHandler Is Needed

1. Prevent Server Freezing

Operations like:

```
readLine();
```

are blocking.

Without separate handlers:

- One waiting client can block the entire server.

With handlers:

- Each client waits independently.
-

2. Keep the Server Listening

The main server thread immediately returns to:

```
serverSocket.accept();
```

and continues accepting new connections.

3. Fault Isolation

If one client disconnects:

- Only that client's handler stops.
 - Other clients remain unaffected.
 - Server continues running.
-

4. Independent Client State

Each handler maintains:

- Socket
- Input stream
- Output stream
- Client-specific information

independently.

Important Concepts Learned

Blocking I/O

Operations that pause execution until data becomes available.

Example:

```
readLine();
```

Multi-threading

Running multiple tasks concurrently using separate threads.

Runnable

Java interface used to define executable tasks.

ExecutorService

Framework for managing and reusing threads efficiently.

Thread Pool

Collection of reusable worker threads.

Result

✓ Multiple clients can communicate with the server simultaneously.

Task 4: Implement Broadcasting System

Goal

Allow the server to send a message to every connected client.

Changes Made in Server.java

1. Added Thread-Safe Client Registry

```
CopyOnWriteArrayList<ClientHandler> activeClients;
```

Purpose

Maintains a list of all connected clients.

Why It Is Needed

- Tracks active connections.
 - Prevents concurrent modification issues.
 - Allows safe access from multiple threads.
-

2. Added Separate Console Input Thread

```
new Thread(() -> {  
    // server console input  
}).start();
```

Problem Solved

The infinite:

```
serverSocket.accept();
```

loop blocks execution.

Without a separate thread:

- Server cannot read keyboard input while accepting clients.

Benefit

The server can:

- Accept client connections.
- Read administrator messages.

simultaneously.

3. Implemented Broadcast Loop

```
for(ClientHandler client : activeClients){  
    client.sendMessage(message);  
}
```

Purpose

Sends a message to every connected client.

Benefit

One server message reaches all active clients instantly.

Changes Made in ClientHandler.java

1. Added sendMessage() Method

```
public void sendMessage(String msg)
```

Purpose

Provides access to the client's output stream.

Benefit

Allows the server to send messages directly to connected clients.

2. Added Automatic Client Registration

```
activeClients.add(this);
```

Purpose

Registers newly connected clients.

Benefit

New clients become immediately available for broadcasting.

3. Added Automatic Client Cleanup

```
activeClients.remove(this);
```

Placed inside a finally block.

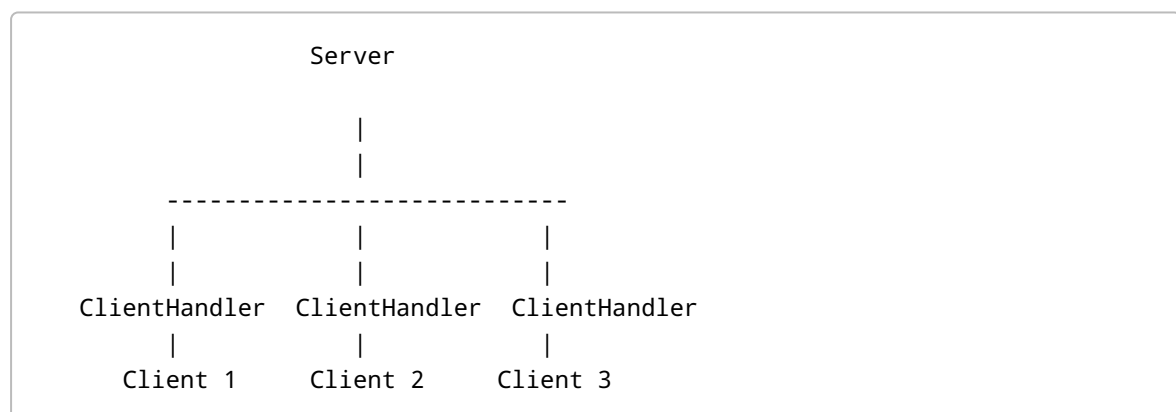
Purpose

Removes disconnected clients from the registry.

Benefit

- Prevents dead connections.
 - Avoids broadcasting errors.
 - Keeps the active client list accurate.
-

System Architecture Achieved



Workflow

1. Server continuously listens for connections.
 2. Each client gets a dedicated `ClientHandler`.
 3. `ClientHandlers` run inside a thread pool.
 4. Active clients are stored in a shared registry.
 5. Server broadcasts messages to all connected clients.
-

Achievements Completed

- ✓ Converted single-client architecture into multi-client architecture
 - ✓ Implemented continuous connection acceptance
 - ✓ Learned blocking I/O concepts
 - ✓ Implemented `ClientHandler` design pattern
 - ✓ Added `ExecutorService` thread pool
 - ✓ Used `Cached Thread Pool` for thread reuse
 - ✓ Enabled concurrent client communication
 - ✓ Implemented thread-safe client registry
 - ✓ Added server-side broadcasting
 - ✓ Implemented automatic client registration
 - ✓ Implemented automatic client cleanup
 - ✓ Built a scalable foundation for a chat application
-

Key Technologies and Concepts Learned

- Java Sockets
- `ServerSocket`
- `Socket`
- Multi-threading
- `Runnable` Interface
- `ExecutorService`
- `Cached Thread Pool`
- Blocking I/O
- Concurrent Programming
- `CopyOnWriteArrayList`

- Broadcast Communication
 - Client Lifecycle Management
 - Thread-Safe Collections
-

Next Development Goal

Implement full client-to-client communication through the server.

Target Features:

- Client messages forwarded to other clients.
 - Usernames for connected clients.
 - Private messaging.
 - Online user list.
 - Chat room functionality.
 - Dockerization.
 - Cloud deployment (AWS).
 - Peer-to-peer communication experiments.
-

End of Day Summary

Today I successfully transformed a simple one-client chat server into a multi-client server architecture using `ClientHandler` threads and `ExecutorService` thread pooling. I also implemented a broadcasting mechanism that allows the server to communicate with all connected clients simultaneously. This established the foundation for building a scalable real-time chat system.

Project Development Journal

Day 02 — 15 June 2026

Project Goal

Enhance the chat application by:

- Introducing usernames for clients.
 - Enabling client-to-client communication.
 - Adding private messaging functionality.
 - Preventing username collisions by enforcing unique usernames.
-

Initial State

At the start of Day 02:

- Multiple clients could connect to the server.
 - The server could broadcast messages.
 - Clients could send messages only to the server.
 - Clients were identified using IP addresses.
 - No private messaging existed.
 - Duplicate usernames were allowed.
-

Task 1: Add Usernames to Clients

Problem

Clients were identified using their IP addresses.

This created several issues:

- IP addresses are not user-friendly.
 - Users cannot easily identify who sent a message.
 - Private messaging becomes difficult to implement.
-

Solution

Added username support inside `ClientHandler`.

Added Username Variable

```
private String username;
```

This variable stores the client's chosen username.

Added Username Getter

```
public String getUsername() {  
    return this.username;  
}
```

Purpose

Allows other classes, especially the server, to access the client's username.

Updated Message Formatting

Replaced IP address references with usernames throughout the application.

Before:

```
[192.168.1.10]: Hello
```

After:

```
[Mahesh]: Hello
```

Result

✅ Every client now has a human-readable identity.

Task 2: Enable Client-to-Client Messaging

Problem

Clients could send messages to the server, but other connected clients could not see those messages.

The chat system lacked peer communication.

Solution

Implemented a new method:

```
private void broadcastMessage(String messageToSend)
```

Purpose

Distributes a client's message to all connected users.

Workflow

Step 1: Iterate Through Active Clients

Loop through:

```
Server.activeClients
```

which contains all connected users.

Step 2: Filter the Sender

Check whether the current client in the loop is the original sender.

```
!client.username.equals(this.username)
```

Step 3: Deliver Message

Send the message to every other connected client.

```
client.sendMessage(messageToSend);
```

Why Exclude the Sender?

If the sender receives the same message back:

- Duplicate messages may appear.
- Extra network traffic is generated.
- User experience becomes confusing.

Therefore, the sender is filtered out.

Result

- ✓ Clients can now communicate with all connected users.
 - ✓ Chatroom-style communication is achieved.
-

Task 3: Implement Private Messaging

Problem

All messages were visible to every connected user.

There was no way to have a private conversation.

Solution

Created a dedicated private messaging method.

```
private void sendPrivateMessage(  
    String targetUser,  
    String messageToSend) {  
  
    boolean found = false;  
  
    for (ClientHandler client : Server.activeClients) {  
  
        if (client.username.equalsIgnoreCase(targetUser)) {  
  
            client.sendMessage(  
                "[PM from " + this.username + "]: "  
                + messageToSend  
            );  
  
            found = true;  
        }  
    }  
}
```

```
        break;
    }
}

if (!found) {
    this.sendMessage(
        "[SYSTEM ERROR]: User '"
        + targetUser
        + "' not found."
    );
}
```

How It Works

Step 1

Loop through all connected clients.

Step 2

Search for a username matching the target user.

Step 3

Send the message directly to that user's socket.

Step 4

If no user is found, notify the sender.

Private Message Command

Users must use:

```
/pm <username> <message>
```

Example:

```
/pm Rahul Hello, are you available?
```

Command Processing Logic

```
if (inputLine.trim().startsWith("/pm ")) {

    String[] tokens =
        inputLine.trim().split(" ", 3);

    if (tokens.length >= 3) {

        String targetUser = tokens[1];
        String privateMsg = tokens[2];

        sendPrivateMessage(
            targetUser,
            privateMsg
        );

    } else {

        this.sendMessage(
            "[SYSTEM]: Invalid format. Use: "
            + "/pm <username> <message>"
        );

    }

} else {

    broadcastMessage(
        "[" + this.username + "]: "
        + inputLine
    );

}
```

Result

- ✅ Users can send private messages.
 - ✅ Private messages are visible only to the intended recipient.
 - ✅ Invalid commands are handled gracefully.
-

Task 4: Prevent Username Collisions

Problem

Two clients could join using the same username.

Example:

Mahesh
Mahesh

This causes:

- User identity confusion.
- Private messaging errors.
- Message ownership ambiguity.

Solution

Implemented username uniqueness validation.

Added Validation Method

```
public boolean isUsernameTaken(String name) {  
    for (ClientHandler client :  
        Server.activeClients) {  
        if (client.username  
            .equalsIgnoreCase(name)) {  
            return true;  
        }  
    }  
    return false;  
}
```

Purpose

Checks whether a username already exists among connected clients.

Username Registration Workflow

When a client connects:

1. Request username.
 2. Validate username.
 3. Reject invalid usernames.
 4. Repeat until a unique username is entered.
-

Username Input Loop

```
while (true) {

    out.println(
        "SUBMIT_NAME_REQ: Please enter a username:"
    );

    String inputName = in.readLine();

    if (inputName == null) {
        return;
    }

    inputName = inputName.trim();

    if (inputName.isEmpty()) {

        out.println(
            "[SYSTEM ERROR]: Username cannot be blank!"
        );

    } else if (isUsernameTaken(inputName)) {

        out.println(
            "[SYSTEM ERROR]: That username is already taken! Try another."
        );

    } else {

        this.username = inputName;
        break;
    }
}
```

Validation Rules

Username Cannot Be Empty

✗ Invalid

""

Username Must Be Unique

✗ Invalid

Mahesh
Mahesh

Unique Username Accepted

✓ Valid

Mahesh
Rahul
Amit

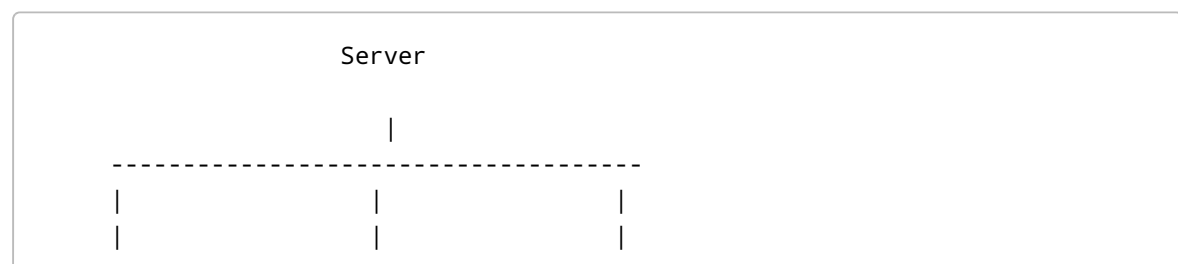
Result

✓ Duplicate usernames are prevented.

✓ Every connected client has a unique identity.

✓ Private messaging becomes reliable.

System Architecture Achieved



Mahesh	Rahul	Amit
Public Messages (Broadcast)		
Mahesh	---	Everyone
Rahul	---	Everyone
Amit	---	Everyone
Private Messages		
Mahesh	---	Rahul
Rahul	---	Amit
Amit	---	Mahesh

Achievements Completed

- ✓ Added usernames to clients
 - ✓ Replaced IP-based identification
 - ✓ Implemented client-to-client messaging
 - ✓ Added broadcast functionality
 - ✓ Implemented private messaging
 - ✓ Added `/pm` command support
 - ✓ Added user lookup mechanism
 - ✓ Added username uniqueness validation
 - ✓ Prevented username collisions
 - ✓ Improved chat usability and reliability
-

Key Concepts Learned

- User Identity Management
- Broadcast Communication
- Private Messaging Systems
- Command Parsing
- Input Validation
- Username Collision Handling
- Collection Traversal

- Socket-Based Direct Communication
 - Client Discovery Mechanisms
 - Chat Application Architecture
-

Next Development Goal

Planned improvements:

- Online user listing command.
 - User join/leave notifications.
 - Message timestamps.
 - Chat history support.
 - Group chat rooms.
 - Persistent user sessions.
 - Docker containerization.
 - AWS deployment.
-

End of Day Summary

Today I transformed the application from a basic broadcast-based communication system into a functional chat platform. I introduced usernames for user identification, implemented public client-to-client messaging, added private messaging through the `/pm` command, and enforced unique usernames to prevent identity conflicts. These improvements significantly enhanced both usability and system reliability while moving the project closer to a complete real-time chat application.

Project Development Journal

Day 03 — 16 June 2026

Project Goal

Today was focused on:

- Organizing and cleaning the project structure.
 - Publishing the latest version to GitHub.
 - Creating project documentation through a README file.
 - Beginning Docker learning and understanding containerization fundamentals.
-

Initial State

At the start of Day 03:

The chat application already supported:

- Multiple client connections.
- Username-based identification.
- Client-to-client communication.
- Public message broadcasting.
- Private messaging using `/pm`.
- Unique username validation.
- Thread pool-based concurrency.

The application was functionally complete for its current stage, but project organization and deployment knowledge needed improvement.

Task 1: Project Cleanup and GitHub Update

Objective

Prepare the project repository in a more professional and maintainable format.

Actions Performed

Cleaned Project Structure

Reorganized project files and folders.

Goals:

- Improve readability.
 - Remove unnecessary files.
 - Follow a cleaner project hierarchy.
 - Make the project easier to understand for other developers.
-

Updated Existing GitHub Repository

Instead of creating a new repository:

- Used the existing project repository.
 - Replaced outdated files with the latest implementation.
 - Pushed all recent features and improvements.
-

Added README Documentation

Created a README file describing:

- Project overview.
 - Features.
 - Technologies used.
 - Setup instructions.
 - Future enhancements.
-

What I Learned

Importance of Project Structure

A project is not only about writing code.

Good organization:

- Makes maintenance easier.
 - Improves collaboration.
 - Helps recruiters understand the project quickly.
 - Makes debugging easier.
-

Importance of README Files

README acts as the project's front page.

It helps users understand:

- What the project does.

- How to run it.
 - Technologies used.
 - Future plans.
-

Result

- ✓ Project structure cleaned.
 - ✓ Existing GitHub repository updated.
 - ✓ README file added.
 - ✓ Project became more professional and presentable.
-

Task 2: Docker Exploration

Objective

Start learning Docker to prepare the application for future containerization and deployment.

Today was primarily dedicated to understanding Docker fundamentals.

Topic 1: What is Docker?

Understanding

Docker is a containerization platform that packages an application along with all its dependencies into a single portable unit.

This ensures:

- Consistent execution across environments.
 - Easier deployment.
 - Simplified dependency management.
-

Why Docker Exists

Without Docker:

Works on my machine ✗
Fails on another machine ✗

Dependency conflicts ✗
Environment mismatch ✗

With Docker:

Runs the same everywhere ✓
Consistent environment ✓
Easy deployment ✓
Portable applications ✓

Topic 2: What is a Docker Image?

Understanding

A Docker Image is a blueprint or template used to create containers.

Think of it as:

Class -> Object
Image -> Container

Characteristics

- Read-only template.
- Contains application code.
- Contains dependencies.
- Contains runtime configuration.

Example

For a Java application, an image may contain:

Operating System
Java Runtime
Application Code
Required Libraries

all bundled together.

What I Learned

A Docker Image does not run by itself.

It is only a template used to create running containers.

Topic 3: What is a Docker Container?

Understanding

A Container is a running instance of a Docker Image.

Example:

```
Image = Blueprint
```

```
Container = Running Application
```

Real-World Analogy

```
Recipe = Image
```

```
Cooked Food = Container
```

The recipe describes how to make the food.

The cooked food is the actual running result.

Characteristics

- Lightweight
 - Isolated
 - Portable
 - Fast startup
 - Uses fewer resources than virtual machines
-

What I Learned

Multiple containers can be created from a single image.

Example:

```
Chat Server Image
```

```
|  
|----> Container 1  
|----> Container 2  
|----> Container 3
```

Topic 4: How to Run a Container

Basic Workflow

Step 1

Pull an image.

Example:

```
docker pull hello-world
```

Step 2

Run a container.

Example:

```
docker run hello-world
```

Step 3

Docker creates a container from the image and executes it.

Understanding the Flow

```
Docker Hub  
↓  
Pull Image  
↓
```

```
Local Machine
  ↓
docker run
  ↓
Container Created
  ↓
Application Executes
```

Important Commands Explored

View Images

```
docker images
```

Displays all downloaded images.

View Running Containers

```
docker ps
```

Displays currently running containers.

View All Containers

```
docker ps -a
```

Displays both active and stopped containers.

Run a Container

```
docker run <image-name>
```

Creates and starts a container.

Result

- ✅ Understood Docker workflow.
- ✅ Learned the relationship between images and containers.

- ✓ Learned how containers are created and executed.
-

Key Concepts Learned Today

Software Engineering

- Project Organization
- Repository Maintenance
- Documentation Practices
- README Creation

Docker Fundamentals

- Docker
 - Containerization
 - Docker Images
 - Docker Containers
 - Docker Hub
 - Image Lifecycle
 - Container Lifecycle
 - Basic Docker Commands
-

Challenges Faced

Docker Installation

Spent time understanding:

- Docker Desktop installation.
 - Required system settings.
 - Virtualization support.
 - Initial Docker setup process.
-

Achievements Completed

- ✓ Cleaned project structure
- ✓ Updated GitHub repository
- ✓ Added README documentation
- ✓ Installed Docker
- ✓ Understood Docker fundamentals

✓ Learned Images vs Containers

✓ Learned container execution workflow

✓ Explored basic Docker commands

Next Development Goal

Planned tasks for Day 04:

- Create a Dockerfile for the chat application.
 - Build a custom Docker image.
 - Run the Java chat server inside a container.
 - Understand Docker networking.
 - Learn port mapping.
 - Run multiple containers simultaneously.
-

End of Day Summary

Today was focused less on coding features and more on improving project quality and learning deployment technologies. I organized the project structure, updated the GitHub repository, created documentation through a README file, and began learning Docker fundamentals. I developed a clear understanding of Docker Images, Containers, and the basic workflow used to package and run applications in isolated environments. This knowledge will be essential for containerizing and deploying the chat application in future stages.

Project Development Journal

Day 04 — 17 June 2026

Project Goal

Containerize the Java Chat Application using Docker and understand how multiple server instances can be deployed using containers.

Initial State

At the start of Day 04:

The application already supported:

- Multiple client connections
- Usernames
- Broadcast messaging
- Private messaging
- Unique username validation
- Thread pool architecture

The application was running directly on the host machine and had not yet been containerized.

Task 1: Create a Dockerfile

Objective

Package the Java chat server into a Docker image so it can run consistently on any machine with Docker installed.

Dockerfile Created

```
``dockerfile id="wfj3wr" FROM eclipse-temurin:21-jdk
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN javac Server.java ClientHandler.java
```

```
EXPOSE 5000
```

CMD ["java", "Server"]

```
---  
  
## Dockerfile Explanation  
  
### 1. Base Image  
  
```dockerfile id="f09d11"  
FROM eclipse-temurin:21-jdk
```

Purpose:

- Uses Java 21 environment.
  - Provides JDK required to compile and run Java code.
- 

## 2. Working Directory

```
```dockerfile id="7mklm6" WORKDIR /app
```

```
Purpose:  
  
- Creates a dedicated directory inside the container.  
- All subsequent commands execute inside this folder.  
  
---  
  
### 3. Copy Project Files  
  
```dockerfile id="5s53fu"  
COPY . .
```

Purpose:

- Copies all project files from the local machine into the container.
- 

## 4. Compile Java Source Code

```
```dockerfile id="vkg2xg" RUN javac Server.java ClientHandler.java
```

```
Purpose:  
  
- Compiles Java source files.  
- Generates `.class` files during image creation.
```

5. Expose Application Port

```
```dockerfile id="svy7b9"  
EXPOSE 5000
```

Purpose:

- Documents that the application listens on port 5000.
- Helps when configuring container networking.

---

## 6. Startup Command

```
```dockerfile id="njlwmn" CMD ["java", "Server"]
```

Purpose:

- Executes automatically when the container starts.
- Launches the Java chat server.

What I Learned

A Dockerfile acts as a blueprint that describes:

- Environment setup
- Dependencies
- Build instructions
- Startup behavior

for a containerized application.

Result

✅ Successfully created the first Dockerfile for the project.

Task 2: Build Docker Image

Objective

Convert the Dockerfile into a reusable Docker image.

```
---
```

```
## Build Command
```

```
docker build -t java-messenger-server .
```

Breakdown

docker build

Builds an image.

-t

Assigns a tag (name) to the image.

java-messenger-server

Custom image name.

.

Uses the current directory as the build context.

What Happened

Docker:

1. Read the Dockerfile.
2. Downloaded the Java base image.
3. Copied project files.
4. Compiled Java code.
5. Created a reusable image.

Result

✅ Docker image successfully created.

Task 3: Verify Docker Image

Command

docker images

```
---

## Purpose

Displays all locally available Docker images.

Information shown:

- Repository name
- Tag
- Image ID
- Creation date
- Image size

---

## Result

✅ Verified the image was successfully created.

---

# Task 4: Run the Container

## Objective

Launch the chat server inside a Docker container.

---

## Command

```bash id="z6h6jl"
docker run -p 5000:5000 java-messenger-server
```

---

## Understanding Port Mapping

Host Port : Container Port

5000 : 5000

Meaning:

- Requests arriving on host port 5000
- Are forwarded to port 5000 inside the container

---

## ## What Happened

Docker:

1. Created a container from the image.
2. Started the Java server.
3. Opened port 5000.
4. Began accepting client connections.

---

## ## Result

- ✓ Chat server successfully running inside Docker.

---

## # Task 5: Test Containerized Server

### ## Objective

Verify that clients can connect to the server running inside a container.

---

### ## Testing Process

#### ### Step 1

Keep the Docker container running.

---

#### ### Step 2

Open a second terminal.

---

#### ### Step 3

Run the client application.

```
```bash id="i8pf0s"
java Client
```

Observation

Client successfully connected to the Dockerized server.

Messages were transmitted correctly.

Result

✓ Dockerized server functioning correctly.

✓ Client-server communication verified.

Task 6: Server Architecture Improvement

Observation

While testing, I noticed that my server had the ability to send chat messages.

This design worked technically, but after researching real-world chat architectures, I learned that servers typically do not participate in conversations.

Understanding

A chat server's primary responsibility is:

- Accept connections.
- Route messages.
- Manage users.
- Handle communication logic.

The server itself is not usually a chat participant.

Changes Made

Removed the server-as-user behavior.

Focused the server exclusively on:

- Message routing
 - User management
 - Communication handling
-

What I Learned

Separation of Responsibilities

A well-designed system assigns clear responsibilities.

Server:

``text id="vhvg08" Connection Management User Management Message Routing

Clients:

```
``text id="ak6t1n"
Send Messages
Receive Messages
Interact With Users
```

Result

✓ Cleaner architecture.

✓ More realistic chat server design.

Task 7: Deploy Multiple Server Instances

Objective

Explore Docker's ability to run multiple copies of the same application.

Understanding

Since the server is packaged inside an image, Docker can create multiple containers from the same image.

Server Instance 1

```
```bash id="0zdkpk" docker run -d -p 5001:5000
--name server1
java-messenger-server
```

---

## Server Instance 2

```
```bash id="wj1wmn"
docker run -d -p 5002:5000
--name server2
java-messenger-server
```

Server Instance 3

```
```bash id="74t2h2" docker run -d -p 5003:5000
--name server3
java-messenger-server
```

---

## Port Mapping Explanation

```
```text id="ejx0rf"
Host Port -> Container Port
```

```
5001 -> 5000
5002 -> 5000
5003 -> 5000
```

Although every container listens internally on:

```
```text id="fq7um3" 5000
```

Docker maps them to different host ports.

---

## Important Understanding

Inside every container:

```
```text id="c2o1yn"
"I'm listening on port 5000"
```

Docker handles the translation externally.

This allows multiple copies of the same application to run simultaneously.

What I Learned

Docker containers are isolated environments.

Multiple containers can:

- Use the same code.
- Use the same internal port.
- Run independently.

as long as Docker maps them to different host ports.

Result

✅ Successfully launched multiple server instances.

✅ Learned Docker port mapping.

✅ Understood container isolation.

Task 8: Verify Running Containers

Command

```
```bash id="jq4q7f" docker ps
```

```

Purpose

Displays currently running containers.

Information shown:

- Container ID
- Image
- Status
```



```

- Ports
- Container Name

Result

✅ Verified all running server containers.

Architecture Achieved

```text id="aj6wlu"
    Docker Host

    -----
    |                               |
    |                               |
Server1          Port 5001
Server2          Port 5002
Server3          Port 5003
    |                               |
    |                               |
    -----

    Same Docker Image

    java-messenger-server

```

Achievements Completed

- ✅ Created Dockerfile
- ✅ Built custom Docker image
- ✅ Verified image creation
- ✅ Ran chat server inside a container
- ✅ Tested client-container communication
- ✅ Improved server architecture
- ✅ Learned Docker networking basics
- ✅ Learned port mapping

✓ Created multiple server instances

✓ Verified running containers

Key Concepts Learned

Docker Concepts

- Dockerfile
- Docker Image
- Docker Container
- Image Build Process
- Container Lifecycle
- Port Mapping
- Container Isolation
- Container Deployment

Software Architecture Concepts

- Separation of Concerns
 - Message Routing
 - Server Responsibilities
 - Scalable Deployment
-

Next Development Goal

Planned topics for future development:

- Docker Compose
 - Multi-container networking
 - Container logs
 - Persistent volumes
 - AWS deployment
 - Reverse proxy configuration
 - Load balancing
 - Container orchestration concepts
-

End of Day Summary

Today I successfully containerized my Java Chat Application using Docker. I created a Dockerfile, built a custom image, and launched the server inside a container. After verifying client connectivity, I refined the system architecture by removing server-side messaging responsibilities and focusing solely on message routing. I also explored Docker's scalability by running multiple independent server instances from the same image and learned how Docker port mapping enables multiple containers to coexist on a single machine.